CONTENTS

- Overview
1 Prompt Caching Is Not Output Caching
2 Inside the Transformer: KV Pairs and the Prefill Phase
3 Why Caching Pays Off with Long, Reused Context
4 What Gets Cached in Practice
5 Prefix Matching: How Caching Decides What to Reuse
6 Practical Limits: Minimum Size, Expiration, and Explicit Caching
- Glossary
- Check yourself
- Where to go next

Intermediate | LLM inference optimization | Condensed | IBM Technology

What is Prompt Caching? Optimize LLM Latency with AI Transformers

Understand how caching the input to an LLM — not its output — cuts cost and latency on long, repeated prompts.

For developers building LLM applications who want to cut API latency and cost on prompts with large, repeated context · 27 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video | Read the full lesson | Read the highlights

BEFORE YOU START

- Basic familiarity with how you send a prompt to an LLM through an API (input in, generated text out)
- A rough idea of what a transformer model and 'tokens' are

Overview

Prompt caching is one of the simplest ways to make large language model applications faster and cheaper, but its name is misleading. It does not cache the model's answer. It caches the internal computation the model does while reading your input, so that the next time the same input (or the same beginning of it) shows up, the model can skip straight to generating a response instead of re-reading everything from scratch.

To understand why that works, you need a rough picture of what an LLM does before it writes its first output token: it converts your prompt into a set of internal representations called key-value (KV) pairs, one set per transformer layer, and that conversion is where most of the pre-generation cost lives. Prompt caching stores those KV pairs so repeated or shared prefixes never have to be recomputed.

This lesson walks through the distinction between prompt caching and output caching, what actually gets cached and why, how prompt structure determines whether caching kicks in at all, and the practical limits — minimum size, expiration, automatic versus explicit caching — that decide whether it is worth using for a given application.

KEY TAKEAWAYS

- ✓ Prompt caching stores the model's internal representation of the input (KV pairs), not the output — every cached request still generates a fresh response.
- ✓ This is different from output caching, which stores the final answer and skips the model call entirely for identical repeated queries.
- ✓ Before generating any output, a transformer computes key-value pairs for every token at every layer; this is called the prefill phase, and it is the part prompt caching speeds up.
- ✓ Caching only pays off once a prompt is large — short prompts finish the prefill phase so fast that managing a cache costs more than it saves.
- ✓ Common things to cache are large reference documents, system prompts, few-shot examples, tool/function definitions, and conversation history — anything static that gets reused across requests.
- ✓ Automatic caching relies on prefix matching: it compares the new prompt to the cached one token by token from the start and stops at the first difference, so static content must come before dynamic content (like the user's question) or the cache never gets used.
- ✓ Most systems require a prompt to be around 1024 tokens or more before caching activates, and cached entries typically expire after 5 to 10 minutes (some last up to 24 hours) to keep data fresh.
- ✓ Some providers cache automatically based on prefix matching; others require you to explicitly mark which segments of a prompt should be cached in the API call.

01 Prompt Caching Is Not Output Caching

0:00

> Prompt caching stores what the model computed from your input, not the answer it produced — every request still gets a freshly generated response.

KEY POINTS

- → Output caching stores the final answer and returns it verbatim for repeated identical prompts, skipping the model entirely.
- → Prompt caching stores only the model's internal processing of the input; it still generates a new response every time.
- → Because prompt caching does not require an identical full prompt, it works even when only part of the prompt (e.g., a shared document) repeats and the rest (the question) changes.

Database cache vs. prompt cache

A SQL query cache stores 'SELECT * FROM orders WHERE id=42' → its result row, and returns that row instantly on the next identical query. An LLM prompt cache instead stores the model's internal representation of a 50-page manual pasted into the prompt; if the next request pastes the same manual but asks a different question, the manual's representation is reused but the model still generates a brand-new answer to the new question.

02 Inside the Transformer: KV Pairs and the Prefill Phase

2:07

Before an LLM can write its first output token, it converts every input token into key-value pairs at every transformer layer — that conversion is the expensive step prompt caching skips.

KEY POINTS

- → KV pairs are the model's internal representation of how tokens in the prompt relate to each other; they are computed at every transformer layer.
- → The stage where KV pairs are computed, before any output is generated, is called the prefill phase.
- → Prefill cost grows with prompt length and model depth (number of layers), which is why it matters most for long prompts.
- → Prompt caching stores the prefill output (KV pairs) so a repeated prefix does not need to be recomputed.

Prefill cost at different prompt sizes

A prompt like 'What's the capital of France?' is only a handful of tokens, so prefill finishes in a tiny fraction of a second — caching it saves almost nothing. A prompt containing a 50-page document is thousands of tokens; computing KV pairs for all of them across every transformer layer is millions of operations, so caching that prefill output produces a noticeable latency drop on the next request that reuses the same document.

03 Why Caching Pays Off with Long, Reused Context

3:36

Caching a large, unchanging block of context — like a document pasted into the prompt

— lets later requests skip straight to processing only the new question.

KEY POINTS

- The pattern that benefits most from prompt caching is: large static content + small changing instruction.
- A cache hit means only the new, non-matching part of the prompt needs to go through prefill.
- Savings accumulate across every subsequent request that reuses the same cached prefix, not just the second one.

Same document, two different requests

Request 1: [50-page manual] + 'Summarize this document.' Request 2 (sent later): [same 50-page manual] + 'What does section 4 say about liability?' On request 2, the system matches the manual against the cache, reuses its KV pairs, and only computes prefill for the new question — instead of reprocessing all 50 pages again.

04 What Gets Cached in Practice

5:05

Anything static and reused — reference documents, system prompts, few-shot examples, tool definitions, and conversation history — are the usual candidates for caching.

KEY POINTS

- System prompts are the most common thing cached, since they are identical on nearly every request a chatbot handles.
- Long reference documents that get queried repeatedly (manuals, contracts, papers) are strong caching candidates.
- Few-shot examples and tool/function definitions are typically static and therefore cacheable.
- Conversation history in a multi-turn chat can be cached up through the last completed turn, leaving only the newest message to reprocess.

A cacheable system prompt

A customer-service chatbot's instructions rarely change between requests, which is exactly what makes them worth caching — the cost of processing this block is paid once, not on every user message.

```
SYSTEM PROMPT (static, cached):
"You are a helpful customer service agent for Acme Corp.
Always be polite and concise. Never discuss competitors.
If you don't know an answer, offer to escalate to a human agent."

USER MESSAGE (dynamic, not cached):
"What's your return policy on electronics?"
```

05 Prefix Matching: How Caching Decides What to Reuse

5:47

> Automatic caching compares a new prompt against the cached one token by token from the very start, and stops reusing the cache the instant a token differs — so prompt order determines whether caching helps at all.

KEY POINTS

- → Prefix matching checks a new prompt against the cache token by token, starting from position zero.
- → The first mismatched token ends the cache hit; everything from that point on is processed as new.
- → Static, reused content (instructions, documents, examples) belongs at the start of the prompt; the dynamic, per-request content (the user's question) belongs at the end.
- → Putting dynamic content first defeats caching entirely, even if most of the prompt is otherwise unchanged.

Cache-friendly vs. cache-breaking prompt order

The first structure keeps the changing question at the end, so the long static prefix (instructions, manual, examples) matches the cache on every call. The second structure puts the question first, so the mismatch happens immediately and nothing after it is ever served from cache.

```
# Cache-friendly (question last)
[system instructions]
[20-page manual]
[few-shot examples]
[user question: "What are the warranty terms?"]
# -> next call with a different question still
#    matches everything before the question

# Cache-breaking (question first)
[user question: "What are the warranty terms?"]
[system instructions]
[20-page manual]
[few-shot examples]
# -> next call's different question mismatches
#    at token 1, so nothing below it is reused
```

06 Practical Limits: Minimum Size, Expiration, and Explicit Caching

7:58

> Caching only pays off above a minimum prompt size, cached entries expire after minutes to hours, and some systems require you to mark cacheable sections explicitly rather than relying on automatic prefix matching.

KEY POINTS

- → Caching typically needs a prompt of at least ~1024 tokens before it produces a net benefit; below that, overhead outweighs savings.
- → Cache entries usually expire after 5 to 10 minutes, though some can persist up to 24 hours.
- → Short expiration windows mean caching helps most for clusters of related requests close together in time.
- → Automatic caching relies purely on prefix matching; explicit caching requires you to mark cacheable segments yourself in the API request.

Explicit vs. automatic caching in an API call

In an automatic-caching system, you send the prompt normally and the provider decides what to cache based on prefix matching against recent requests. In an explicit-caching system, you annotate specific blocks of the request — commonly the system prompt or a large document — so the provider knows exactly what to persist, independent of whether the rest of the prompt matches anything.

```
# Automatic caching: nothing special to write,
# the provider matches prefixes on its own.
response = client.generate(
    prompt=system_instructions + document + question
)

# Explicit caching: mark which blocks are cacheable.
response = client.generate(messages=[
    {"role": "system", "content": system_instructions, "cache": True},
    {"role": "user", "content": document, "cache": True},
    {"role": "user", "content": question}  # not cached, changes every call
])
```

Glossary

Prompt caching

Storing the model's internal processing of an input prompt (or its static prefix) so that repeated or shared portions of future prompts don't need to be recomputed. The model still generates a fresh output every time.

Output caching

Storing the final generated response to a prompt and returning it directly for identical future requests, skipping the model call entirely.

KV pairs (key-value pairs)

Internal values a transformer computes for every token at every layer, representing how each token relates to the others in the input. These are what prompt caching actually stores.

Prefill phase

The stage of LLM processing where the model computes KV pairs for the entire input prompt, before it generates its first output token.

Transformer layer

One of several stacked processing stages in a transformer model; KV pairs are computed independently at each layer as the input passes through the model.

Prefix matching

A technique that compares a new prompt against a cached one token by token from the start, treating everything up to the first differing token as a cache hit.

System prompt

Instructions sent with every request to an LLM that define its behavior, persona, and rules, typically identical across many calls and therefore a strong caching candidate.

Few-shot examples

Sample input/output pairs included in a prompt to show the model the format or style of response desired.

Context window

The maximum amount of text (measured in tokens) an LLM can process as input in a single request.

Token

A chunk of text (roughly a word or part of a word) that an LLM processes as a single unit; prompt length and caching thresholds are measured in tokens.

Check yourself

Answer first, then open each one.

Why does caching provide almost no benefit for a short prompt like 'What's the capital of France?'

The prefill phase for a handful of tokens is already extremely fast, so there is very little computation to save. Below roughly 1024 tokens, the overhead of storing and looking up a cache entry can exceed whatever time it would save, so caching adds cost rather than removing it.

If a prompt caching system claims to speed up repeated requests, what exactly is it reusing — the prompt's processing, or the model's answer? Why does that distinction matter for a document Q&A tool where every question is different?

It reuses the prompt's processing (the cached KV pairs from the prefill phase), not the answer. That matters for a document Q&A tool because every question about the document is different, so output caching (storing answers) would never get a hit — but prompt caching still helps, because the document itself is an unchanging prefix shared across all those different questions.

A developer puts the user's question at the very start of their prompt, followed by a large static system prompt and reference document. Why does this defeat automatic caching even though most of the prompt content doesn't change between requests?

Automatic caching uses prefix matching, which compares tokens from the beginning of the prompt. Since the question changes every request and sits first, the mismatch happens at token one, so the cache lookup fails before it ever reaches the static content behind it — none of that static content gets served from cache, regardless of how much of it is identical to the previous request.

Why do caching systems set an expiration window (typically 5-10 minutes) instead of keeping cached prompt data indefinitely?

A short expiration keeps cached content from going stale — if underlying documents, instructions, or context could change, an indefinitely cached prefix could serve outdated information. It also bounds how much data the system has to store at once, since most caching value comes from bursts of related requests close together in time rather than requests spread far apart.

What's the practical difference between a provider that does automatic caching versus one that requires explicit caching, and what does the developer have to do differently in each case?

With automatic caching, the provider applies prefix matching to every request on its own; the developer's only job is to structure the prompt with static content first so a match is possible. With explicit caching, the developer must actively mark which segments of the prompt (e.g., the system prompt or a document) should be treated as cacheable in the API call itself, giving more control but requiring more deliberate request design.

Where to go next

- → Look up how a specific LLM provider's API documents its prompt caching (minimum token thresholds, cache lifetime, and whether it's automatic or explicit) and compare it against the general figures in this lesson.
- → Measure the latency difference on a real API call by sending the same large document twice with different trailing questions, once with the question first and once with the question last.
- → Read about the companion mechanism, KV caching during multi-turn generation within a single response, to see how it differs from prompt caching across separate requests.
- → Design a system prompt + reference document + few-shot example structure for an application you're building, ordering it to maximize cache hits.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.